# 디코더 아키텍처와 마스킹 메커니즘 (Decoder Architecture & Masking)

## 개요

트랜스포머의 디코더는 인코더의 출력을 받아 순차적으로 토큰을 생성하는 자기회귀적 (autoregressive) 구성 요소입니다. 디코더는 인코더와 유사한 구조를 가지지만, **마스크드 셀프 어텐션**과 **인코더-디코더 어텐션**이라는 두 가지 특별한 메커니즘을 통해 미래 정보를 보지 않으면서도 입력 시퀀스의 정보를 활용할 수 있습니다.

## 디코더의 특수한 구조와 인코더와의 차이점

### 1. 전체 아키텍처 비교

#### 인코더 구조 (복습)

```
입력 → 멀티헤드 셀프 어텐션 → 잔차 연결 & 레이어 정규화 → 피드포워드 → 잔차 연결 & 레이어
```

#### 디코더 구조 (새로운 구조)

```
입력 → 마스크드 멀티헤드 셀프 어텐션 → 잔차 연결 & 레이어 정규화
→ 인코더-디코더 어텐션 → 잔차 연결 & 레이어 정규화
→ 피드포워드 → 잔차 연결 & 레이어 정규화 → 출력
```

### 2. 핵심 차이점 분석

| 구성 요소 | 인코더 | 디코더 | 차이점 |

|---|---|---|---|

| **셀프 어텐션** | 양방향 (모든 위치 참조) | 단방향 (이전 위치만 참조) | 마스킹으로 미래 정보 차단 |

| **크로스 어텐션** | 없음 | 인코더-디코더 어텐션 | 인코더 출력 정보 활용 |

| **생성 방식** | 병렬 처리 | 순차적 생성 | 자기회귀적 특성 |

| **마스킹** | 패딩 마스크만 | 패딩 + 룩어헤드 마스크 | 미래 토큰 접근 방지 |

## 3. 자기회귀적 생성의 필요성

```
def  explain_autoregressive_generation():
"""
자기회귀적 생성이 필요한 이유
언어 생성은 본질적으로 순차적 과정:
- 이전 단어들을 바탕으로 다음 단어 예측
- 미래 정보를 미리 알 수 없음
- 훈련 시에도 이 조건을 시뮬레이션해야 함
"""
generation_process = {
"시작": "<start>",
"1단계": "<start> 안녕",
"2단계": "<start> 안녕 하세요",
"3단계": "<start> 안녕 하세요 <end>",
}
# 각 단계에서 이전 토큰들만 참조 가능
```

```
for step, sequence in generation_process.items():
print(f"{step}: {sequence}")
print(f" → 다음 토큰 예측 시 '{sequence}' 까지만 참조 가능")
return generation_process
```

## 마스크드 셀프 어텐션의 필요성과 구현

### 1. 룩어헤드 마스크(Look-ahead Mask)의 개념

마스크드 셀프 어텐션은 현재 위치보다 뒤에 있는 토큰들을 참조하지 못하도록 차단합니다:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math


def  create_look_ahead_mask(seq_len):
"""
룩어헤드 마스크 생성
Args:
seq_len: 시퀀스 길이
Returns:
mask: [seq_len, seq_len] - 하삼각 행렬 형태의 마스크
"""
```

```
# 하삼각 행렬 생성 (대각선 포함)
mask = torch.tril(torch.ones(seq_len, seq_len))
# 시각화를 위한 예시
print("룩어헤드 마스크 (1=참조가능, 0=차단):")
print(mask.int())
return mask


# 예시 실행
look_ahead_mask = create_look_ahead_mask(5)
```

출력 예시:

```
룩어헤드 마스크 (1=참조가능, 0=차단):
tensor([[1, 0, 0, 0, 0],
[1, 1, 0, 0, 0],
[1, 1, 1, 0, 0],
[1, 1, 1, 1, 0],
[1, 1, 1, 1, 1]])
```

## 2. 마스크드 어텐션 구현

```
class MaskedMultiHeadAttention(nn.Module):
"""
마스크드 멀티헤드 어텐션
```

```
디코더에서 사용되는 특별한 어텐션으로,
미래 토큰을 참조하지 못하도록 마스킹 적용
"""
def  __init__(self, d_model, num_heads, dropout=0.1):
super(MaskedMultiHeadAttention, self).__init__()
assert d_model % num_heads == 0
self.d_model = d_model
self.num_heads = num_heads
self.d_k = d_model // num_heads
# Query, Key, Value 변환 행렬
self.w_q = nn.Linear(d_model, d_model)
self.w_k = nn.Linear(d_model, d_model)
self.w_v = nn.Linear(d_model, d_model)
self.w_o = nn.Linear(d_model, d_model)
self.dropout = nn.Dropout(dropout)
def  scaled_dot_product_attention(self, query, key, value, mask=None):
"""
스케일드 닷-프로덕트 어텐션 (마스킹 포함)
Args:
query, key, value: [batch_size, num_heads, seq_len, d_k]
mask: 어텐션 마스크
Returns:
output: 어텐션 출력
```

```
attention_weights: 어텐션 가중치
"""
d_k = query.size(-1)
# 어텐션 스코어 계산
scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)
# 마스킹 적용 (매우 중요!)
if mask is not None:
# 마스크된 위치에 매우 작은 값 할당 → softmax 후 0에 가까워짐
scores = scores.masked_fill(mask == 0, -1e9)
# 소프트맥스로 확률 분포 변환
attention_weights = F.softmax(scores, dim=-1)
attention_weights = self.dropout(attention_weights)
# 가중합 계산
output = torch.matmul(attention_weights, value)
return output, attention_weights
def  forward(self, query, key, value, mask=None):
"""
마스크드 멀티헤드 어텐션 순전파
Args:
query, key, value: [batch_size, seq_len, d_model]
mask: [batch_size, 1, seq_len, seq_len] 또는 [seq_len, seq_len]
Returns:
output: [batch_size, seq_len, d_model]
```

```
attention_weights: 어텐션 가중치
"""
batch_size, seq_len = query.size(0), query.size(1)
# 1. Linear 변환 및 헤드 분할
Q = self.w_q(query).view(batch_size, seq_len, self.num_heads, self.d_k).transpos
K = self.w_k(key).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(
V = self.w_v(value).view(batch_size, seq_len, self.num_heads, self.d_k).transpos
# 2. 마스크드 어텐션 계산
attention_output, attention_weights = self.scaled_dot_product_attention(Q, K, V,
# 3. 헤드 결합
attention_output = attention_output.transpose(1, 2).contiguous().view(
batch_size, seq_len, self.d_model
)
# 4. 최종 선형 변환
output = self.w_o(attention_output)
return output, attention_weights
```

## 3. 마스킹 효과 시각화

```
def  visualize_masking_effect():
"""마스킹이 어텐션에 미치는 영향 시각화"""
seq_len = 4
d_model = 8
```

```
# 더미 입력 생성
x = torch.randn(1, seq_len, d_model)
# 마스크 없는 일반 어텐션
normal_attention = MaskedMultiHeadAttention(d_model, 2)
normal_output, normal_weights = normal_attention(x, x, x, mask=None)
# 마스크 적용 어텐션
look_ahead_mask = create_look_ahead_mask(seq_len).unsqueeze(0).unsqueeze(0)
masked_output, masked_weights = normal_attention(x, x, x, mask=look_ahead_mask)
print("일반 어텐션 가중치 (모든 위치 참조 가능):")
print(normal_weights[0, 0].detach().numpy().round(3))
print("\n마스크드 어텐션 가중치 (이전 위치만 참조):")
print(masked_weights[0, 0].detach().numpy().round(3))
return normal_weights, masked_weights


# 실행 예시
normal_w, masked_w = visualize_masking_effect()
```

# 인코더-디코더 어텐션 메커니즘 상세 분석

## 1. 크로스 어텐션(Cross-Attention)의 개념

인코더-디코더 어텐션은 디코더가 인코더의 출력 정보를 활용할 수 있게 해주는 메커니즘입니다:

```
class EncoderDecoderAttention(nn.Module):
```

```
"""
인코더-디코더 어텐션 (크로스 어텐션)
특징:
- Query: 디코더의 현재 상태
- Key, Value: 인코더의 출력
- 디코더가 입력 시퀀스의 모든 정보에 접근 가능
"""
def  __init__(self, d_model, num_heads, dropout=0.1):
super(EncoderDecoderAttention, self).__init__()
assert d_model % num_heads == 0
self.d_model = d_model
self.num_heads = num_heads
self.d_k = d_model // num_heads
# Query는 디코더용, Key/Value는 인코더용
self.w_q = nn.Linear(d_model, d_model) # 디코더 입력용
self.w_k = nn.Linear(d_model, d_model) # 인코더 출력용
self.w_v = nn.Linear(d_model, d_model) # 인코더 출력용
self.w_o = nn.Linear(d_model, d_model)
self.dropout = nn.Dropout(dropout)
def  forward(self, decoder_input, encoder_output, encoder_mask=None):
"""
인코더-디코더 어텐션 순전파
Args:
```

```
decoder_input: [batch_size, dec_seq_len, d_model] - 디코더 상태

encoder_output: [batch_size, enc_seq_len, d_model] - 인코더 출력

encoder_mask: 인코더의 패딩 마스크

Returns:

output: [batch_size, dec_seq_len, d_model]

attention_weights: [batch_size, num_heads, dec_seq_len, enc_seq_len]

"""

batch_size = decoder_input.size(0)

dec_seq_len = decoder_input.size(1)

enc_seq_len = encoder_output.size(1)

# Query는 디코더에서, Key/Value는 인코더에서

Q = self.w_q(decoder_input).view(batch_size, dec_seq_len, self.num_heads, self.d

K = self.w_k(encoder_output).view(batch_size, enc_seq_len, self.num_heads, self.

V = self.w_v(encoder_output).view(batch_size, enc_seq_len, self.num_heads, self.

# 어텐션 계산

d_k = Q.size(-1)

scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)

# 인코더 패딩 마스크 적용

if encoder_mask is not None:

scores = scores.masked_fill(encoder_mask == 0, -1e9)

attention_weights = F.softmax(scores, dim=-1)

attention_weights = self.dropout(attention_weights)

output = torch.matmul(attention_weights, V)
```

```
# 헤드 결합
output = output.transpose(1, 2).contiguous().view(
batch_size, dec_seq_len, self.d_model
)
output = self.w_o(output)
return output, attention_weights
```

## 2. 크로스 어텐션의 동작 원리

```
def  explain_cross_attention():
"""크로스 어텐션의 동작 원리 설명"""
# 예시 시나리오: 영어 → 한국어 번역
english_sentence = ["I", "love", "you"]
korean_generation = ["나는", "?"] # "너를" 생성 중
cross_attention_process = {
"Query": {
"출처": "디코더 (한국어 생성 상태)",
"내용": "현재 생성 중인 '?' 위치의 표현",
"역할": "무엇을 찾고 있는지 표현"
},
"Key": {
"출처": "인코더 (영어 입력)",
```

```
"내용": "['I', 'love', 'you']의 표현들",

"역할": "참조 가능한 정보들의 식별자"

},

"Value": {

"출처": "인코더 (영어 입력)",

"내용": "['I', 'love', 'you']의 의미 표현들",

"역할": "실제로 가져올 정보"

},

"결과": {

"어텐션 가중치": "[0.1, 0.1, 0.8]", # "you"에 높은 가중치

"해석": "'?' 위치에서 'you'를 주로 참조",

"생성 결과": "너를"

}

}

return cross_attention_process
```

## 완전한 디코더 레이어 구현

```
class DecoderLayer(nn.Module):

"""

트랜스포머 디코더 레이어

구조:

1. 마스크드 멀티헤드 셀프 어텐션 + 잔차 연결 + 레이어 정규화
```

```
2. 인코더-디코더 어텐션 + 잔차 연결 + 레이어 정규화

3. 피드포워드 네트워크 + 잔차 연결 + 레이어 정규화

"""

def  __init__(self, d_model, num_heads, d_ff, dropout=0.1):

super(DecoderLayer, self).__init__()

# 세 개의 서브레이어

self.masked_self_attention = MaskedMultiHeadAttention(d_model, num_heads, dropou

self.encoder_decoder_attention = EncoderDecoderAttention(d_model, num_heads, dro

self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout)

# 세 개의 레이어 정규화

self.norm1 = nn.LayerNorm(d_model)

self.norm2 = nn.LayerNorm(d_model)

self.norm3 = nn.LayerNorm(d_model)

self.dropout = nn.Dropout(dropout)

def  forward(self, x, encoder_output, look_ahead_mask=None, padding_mask=None):

"""

디코더 레이어 순전파

Args:

x: [batch_size, dec_seq_len, d_model] - 디코더 입력

encoder_output: [batch_size, enc_seq_len, d_model] - 인코더 출력

look_ahead_mask: 룩어헤드 마스크 (미래 토큰 차단)

padding_mask: 패딩 마스크 (패딩 토큰 차단)

Returns:
```

```
output: [batch_size, dec_seq_len, d_model]

self_attention_weights: 셀프 어텐션 가중치

cross_attention_weights: 크로스 어텐션 가중치

"""

# 1. 마스크드 셀프 어텐션

self_attn_output, self_attn_weights = self.masked_self_attention(

x, x, x, mask=look_ahead_mask

)

self_attn_output = self.dropout(self_attn_output)

x = self.norm1(x + self_attn_output) # 잔차 연결 + 정규화

# 2. 인코더-디코더 어텐션

cross_attn_output, cross_attn_weights = self.encoder_decoder_attention(

x, encoder_output, encoder_mask=padding_mask

)

cross_attn_output = self.dropout(cross_attn_output)

x = self.norm2(x + cross_attn_output) # 잔차 연결 + 정규화

# 3. 피드포워드 네트워크

ff_output = self.feed_forward(x)

ff_output = self.dropout(ff_output)

output = self.norm3(x + ff_output) # 잔차 연결 + 정규화

return output, self_attn_weights, cross_attn_weights


# 피드포워드 네트워크 (인코더와 동일)
```

```python
class PositionwiseFeedForward(nn.Module):
"""포지션별 피드포워드 네트워크"""
def __init__(self, d_model, d_ff, dropout=0.1):
super(PositionwiseFeedForward, self).__init__()
self.linear1 = nn.Linear(d_model, d_ff)
self.linear2 = nn.Linear(d_ff, d_model)
self.dropout = nn.Dropout(dropout)
def forward(self, x):
return self.linear2(self.dropout(F.relu(self.linear1(x))))
```

## 자기회귀적 생성 과정과 마스킹 함수

### 1. 완전한 마스킹 함수 구현

```python
def create_masks(src, tgt, pad_token_id=0):
"""
트랜스포머에 필요한 모든 마스크 생성
Args:
src: [batch_size, src_seq_len] - 소스 시퀀스 (인코더 입력)
tgt: [batch_size, tgt_seq_len] - 타겟 시퀀스 (디코더 입력)
pad_token_id: 패딩 토큰 ID
Returns:
src_mask: 인코더 패딩 마스크
```

```
tgt_mask: 디코더 룩어헤드 + 패딩 마스크

src_padding_mask: 크로스 어텐션용 소스 패딩 마스크

"""

# 1. 소스 패딩 마스크 (인코더용)

src_mask = (src != pad_token_id).unsqueeze(1).unsqueeze(1) # [batch, 1, 1, src_l

# 2. 타겟 패딩 마스크

tgt_padding_mask = (tgt != pad_token_id).unsqueeze(1).unsqueeze(1) # [batch, 1,

# 3. 룩어헤드 마스크

tgt_seq_len = tgt.size(1)

look_ahead_mask = torch.tril(torch.ones(tgt_seq_len, tgt_seq_len)).unsqueeze(0).

# 4. 타겟 마스크 = 패딩 마스크 AND 룩어헤드 마스크

tgt_mask = tgt_padding_mask & look_ahead_mask

# 5. 크로스 어텐션용 소스 패딩 마스크

src_padding_mask = src_mask

return src_mask, tgt_mask, src_padding_mask


def  demonstrate_mask_creation():

"""마스크 생성 예시"""

# 예시 데이터

batch_size = 2

src_seq_len = 5

tgt_seq_len = 4

pad_token_id = 0
```

```
# 더미 시퀀스 (0은 패딩)
src = torch.tensor([[1, 2, 3, 4, 0], # 마지막이 패딩
[1, 2, 0, 0, 0]]) # 뒤 3개가 패딩
tgt = torch.tensor([[1, 2, 3, 0], # 마지막이 패딩
[1, 2, 3, 4]]) # 패딩 없음
src_mask, tgt_mask, src_padding_mask = create_masks(src, tgt, pad_token_id)
print("소스 시퀀스:")
print(src)
print("\n소스 마스크 (첫 번째 샘플):")
print(src_mask[0, 0, 0])
print("\n타겟 시퀀스:")
print(tgt)
print("\n타겟 마스크 (첫 번째 샘플):")
print(tgt_mask[0, 0])
return src_mask, tgt_mask, src_padding_mask


# 실행 예시
masks = demonstrate_mask_creation()
```

## 2. 자기회귀적 생성 과정 구현

```
class TransformerDecoder(nn.Module):
```

```
"""완전한 트랜스포머 디코더"""

def  __init__(self, num_layers=6, d_model=512, num_heads=8, d_ff=2048,

vocab_size=10000, max_seq_len=100, dropout=0.1):

super(TransformerDecoder, self).__init__()

self.d_model = d_model

self.num_layers = num_layers

# 임베딩 레이어

self.embedding = nn.Embedding(vocab_size, d_model)

self.positional_encoding = PositionalEncoding(d_model, max_seq_len, dropout)

# 디코더 레이어들

self.layers = nn.ModuleList([

DecoderLayer(d_model, num_heads, d_ff, dropout)

for _ in  range(num_layers)

])

# 최종 출력 레이어

self.final_norm = nn.LayerNorm(d_model)

self.output_projection = nn.Linear(d_model, vocab_size)

def  forward(self, tgt, encoder_output, tgt_mask=None, src_mask=None):

"""

디코더 순전파 (훈련 시)

Args:

tgt: [batch_size, tgt_seq_len] - 타겟 토큰 인덱스

encoder_output: [batch_size, src_seq_len, d_model] - 인코더 출력
```

```
tgt_mask: 타겟 마스크 (룩어헤드 + 패딩)

src_mask: 소스 패딩 마스크

Returns:

output: [batch_size, tgt_seq_len, vocab_size] - 다음 토큰 확률

"""

# 1. 임베딩 + 포지셔널 인코딩

x = self.embedding(tgt) * math.sqrt(self.d_model)

x = self.positional_encoding(x)

# 2. 각 디코더 레이어 통과

all_self_attentions = []

all_cross_attentions = []

for layer in  self.layers:

x, self_attn, cross_attn = layer(x, encoder_output, tgt_mask, src_mask)

all_self_attentions.append(self_attn)

all_cross_attentions.append(cross_attn)

# 3. 최종 정규화 및 출력 투영

x = self.final_norm(x)

output = self.output_projection(x)

return output, all_self_attentions, all_cross_attentions

def  generate(self, encoder_output, src_mask, start_token_id, end_token_id,

max_length=50, temperature=1.0):

"""

자기회귀적 생성 (추론 시)
```

```
Args:
encoder_output: 인코더 출력
src_mask: 소스 마스크
start_token_id: 시작 토큰 ID
end_token_id: 종료 토큰 ID
max_length: 최대 생성 길이
temperature: 생성 다양성 조절
Returns:
generated_sequence: 생성된 토큰 시퀀스
"""
batch_size = encoder_output.size(0)
device = encoder_output.device
# 시작 토큰으로 초기화
generated = torch.full((batch_size, 1), start_token_id, device=device)
for step in  range(max_length - 1):
# 현재까지 생성된 시퀀스에 대한 마스크 생성
tgt_len = generated.size(1)
tgt_mask = torch.tril(torch.ones(tgt_len, tgt_len, device=device))
tgt_mask = tgt_mask.unsqueeze(0).unsqueeze(0)
# 디코더 순전파
with torch.no_grad():
output, _, _ = self.forward(generated, encoder_output, tgt_mask, src_mask)
# 마지막 위치의 로짓에서 다음 토큰 예측
```

```
next_token_logits = output[:, -1, :] / temperature

next_token_probs = F.softmax(next_token_logits, dim=-1)

# 다음 토큰 샘플링

next_token = torch.multinomial(next_token_probs, 1)

# 생성된 시퀀스에 추가

generated = torch.cat([generated, next_token], dim=1)

# 종료 토큰이 생성되면 중단

if (next_token == end_token_id).all():

break

return generated



# 포지셔널 인코딩 (이전 문서에서 구현)

class PositionalEncoding(nn.Module):

"""포지셔널 인코딩"""

def  __init__(self, d_model, max_seq_len=5000, dropout=0.1):

super(PositionalEncoding, self).__init__()

self.dropout = nn.Dropout(dropout)

pe = torch.zeros(max_seq_len, d_model)

position = torch.arange(0, max_seq_len).unsqueeze(1).float()

div_term = torch.exp(torch.arange(0, d_model, 2).float() *

-(math.log(10000.0) / d_model))

pe[:, 0::2] = torch.sin(position * div_term)

pe[:, 1::2] = torch.cos(position * div_term)
```

```
pe = pe.unsqueeze(0)

self.register_buffer('pe', pe)

def  forward(self, x):

x = x + self.pe[:, :x.size(1)]

return  self.dropout(x)
```

## 3. 생성 과정 시각화

```
def  visualize_generation_process():

"""자기회귀적 생성 과정 시각화"""

# 예시: "Hello world" 생성 과정

generation_steps = [

{

"step": 0,

"input": ["<start>"],

"mask": [[1]],

"prediction": "Hello",

"explanation": "시작 토큰만 참조하여 첫 번째 단어 예측"

},

{

"step": 1,

"input": ["<start>", "Hello"],

"mask": [[1, 0], [1, 1]],
```

```
"prediction": "world",
"explanation": "<start>와 Hello를 참조하여 두 번째 단어 예측"
},
{
"step": 2,
"input": ["<start>", "Hello", "world"],
"mask": [[1, 0, 0], [1, 1, 0], [1, 1, 1]],
"prediction": "<end>",
"explanation": "모든 이전 토큰을 참조하여 종료 토큰 예측"
}
]
for step_info in generation_steps:
print(f"Step {step_info['step']}:")
print(f" 입력: {step_info['input']}")
print(f" 마스크: {step_info['mask']}")
print(f" 예측: {step_info['prediction']}")
print(f" 설명: {step_info['explanation']}")
print()
return generation_steps
```

## 완전한 사용 예시

```
def  demonstrate_decoder():
```

```
"""디코더 사용 예시"""

# 하이퍼파라미터

batch_size = 2

src_seq_len = 10

tgt_seq_len = 8

d_model = 512

vocab_size = 1000

# 더미 데이터

src_tokens = torch.randint(1, vocab_size, (batch_size, src_seq_len))

tgt_tokens = torch.randint(1, vocab_size, (batch_size, tgt_seq_len))

# 인코더 출력 (더미)

encoder_output = torch.randn(batch_size, src_seq_len, d_model)

# 마스크 생성

src_mask, tgt_mask, src_padding_mask = create_masks(src_tokens, tgt_tokens)

# 디코더 생성

decoder = TransformerDecoder(

num_layers=6,

d_model=d_model,

num_heads=8,

d_ff=2048,

vocab_size=vocab_size

)

# 훈련 모드 순전파
```

```
print("=== 훈련 모드 (Teacher Forcing) ===")

output, self_attns, cross_attns = decoder(tgt_tokens, encoder_output, tgt_mask,

print(f"출력 크기: {output.shape}") # [batch_size, tgt_seq_len, vocab_size]

print(f"셀프 어텐션 레이어 수: {len(self_attns)}")

print(f"크로스 어텐션 레이어 수: {len(cross_attns)}")

# 생성 모드

print("\n=== 생성 모드 (Autoregressive) ===")

generated = decoder.generate(

encoder_output=encoder_output,

src_mask=src_padding_mask,

start_token_id=1,

end_token_id=2,

max_length=15

)

print(f"생성된 시퀀스 크기: {generated.shape}")

print(f"생성된 토큰들: {generated[0].tolist()}")

return decoder, output, generated


# 실행 예시

decoder, train_output, generated_seq = demonstrate_decoder()
```

## 성능 최적화 및 실무 고려사항

## 1. 메모리 효율적인 생성

```
class EfficientDecoder(TransformerDecoder):
"""메모리 효율적인 디코더"""
def  generate_with_cache(self, encoder_output, src_mask, start_token_id,
end_token_id, max_length=50):
"""
KV 캐시를 사용한 효율적인 생성
매 스텝마다 전체 시퀀스를 다시 계산하지 않고,
이전 스텝의 Key, Value를 재사용
"""
batch_size = encoder_output.size(0)
device = encoder_output.device
# 초기 시퀀스
generated = torch.full((batch_size, 1), start_token_id, device=device)
# KV 캐시 초기화
past_key_values = None
for step in  range(max_length - 1):
# 현재 스텝에서는 마지막 토큰만 처리
if step == 0:
input_ids = generated
else:
input_ids = generated[:, -1:] # 마지막 토큰만
```

```
# 캐시를 사용한 순전파
output, past_key_values = self.forward_with_cache(
input_ids, encoder_output, src_mask, past_key_values
)
# 다음 토큰 예측
next_token_logits = output[:, -1, :]
next_token = torch.argmax(next_token_logits, dim=-1, keepdim=True)
# 시퀀스에 추가
generated = torch.cat([generated, next_token], dim=1)
if (next_token == end_token_id).all():
break
return generated
def  forward_with_cache(self, input_ids, encoder_output, src_mask, past_key_valu
"""KV 캐시를 사용한 순전파"""
# 실제 구현에서는 각 레이어의 Key, Value를 캐시
# 여기서는 개념적 구현만 제시
pass
```

## 2. 빔 서치(Beam Search) 구현

```
def  beam_search_generate(decoder, encoder_output, src_mask, start_token_id,
end_token_id, beam_size=4, max_length=50):
"""
```

```
빔 서치를 사용한 고품질 생성

Args:

decoder: 디코더 모델

encoder_output: 인코더 출력

src_mask: 소스 마스크

start_token_id: 시작 토큰

end_token_id: 종료 토큰

beam_size: 빔 크기

max_length: 최대 길이

Returns:

best_sequences: 최고 점수 시퀀스들

"""

batch_size = encoder_output.size(0)

device = encoder_output.device

# 빔 초기화

beams = [{

'sequence': torch.full((batch_size, 1), start_token_id, device=device),

'score': 0.0,

'finished': False

} for _ in  range(beam_size)]

finished_beams = []

for step in  range(max_length - 1):

candidates = []
```

```
for beam in beams:
if beam['finished']:
continue
# 현재 빔에 대해 다음 토큰 예측
tgt_len = beam['sequence'].size(1)
tgt_mask = torch.tril(torch.ones(tgt_len, tgt_len, device=device))
tgt_mask = tgt_mask.unsqueeze(0).unsqueeze(0)
with torch.no_grad():
output, _, _ = decoder(beam['sequence'], encoder_output, tgt_mask, src_mask)
# 다음 토큰 확률
next_token_logits = output[:, -1, :]
next_token_probs = F.log_softmax(next_token_logits, dim=-1)
# 상위 beam_size개 토큰 선택
top_probs, top_tokens = torch.topk(next_token_probs, beam_size, dim=-1)
for i in  range(beam_size):
new_sequence = torch.cat([beam['sequence'], top_tokens[:, i:i+1]], dim=1)
new_score = beam['score'] + top_probs[0, i].item()
candidates.append({
'sequence': new_sequence,
'score': new_score,
'finished': top_tokens[0, i].item() == end_token_id
})
# 점수 기준으로 정렬하여 상위 beam_size개 선택
```

```
candidates.sort(key=lambda x: x['score'], reverse=True)

# 완료된 빔과 진행 중인 빔 분리

new_beams = []

for candidate in candidates:

if candidate['finished']:

finished_beams.append(candidate)

else:

new_beams.append(candidate)

if len(new_beams) == beam_size:

break

beams = new_beams

# 모든 빔이 완료되면 종료

if not beams:

break

# 최종 결과: 완료된 빔들 중 최고 점수

all_beams = finished_beams + beams

all_beams.sort(key=lambda x: x['score'], reverse=True)

return [beam['sequence'] for beam in all_beams[:beam_size]]
```

## 핵심 개념 정리

### 1. 디코더의 특수한 구조

- **마스크드 셀프 어텐션**: 미래 토큰 참조 방지로 자기회귀적 특성 구현

- **인코더-디코더 어텐션**: 입력 시퀀스 정보를 활용한 조건부 생성
- **3개 서브레이어**: 각각 다른 역할을 수행하는 어텐션과 피드포워드

### 2. 마스킹 메커니즘의 중요성

- **룩어헤드 마스크**: 훈련과 추론 간 일관성 보장
- **패딩 마스크**: 무의미한 패딩 토큰 무시
- **복합 마스크**: 두 마스크의 논리곱으로 완전한 마스킹

### 3. 자기회귀적 생성

- **순차적 처리**: 이전 토큰들을 바탕으로 다음 토큰 예측
- **Teacher Forcing**: 훈련 시 정답 시퀀스 사용으로 효율적 학습
- **추론 시 생성**: 실제 생성된 토큰을 사용한 단계별 생성

## 다음 단계

디코더 아키텍처를 이해했다면, 다음 문서에서는 **피드포워드 네트워크**의 상세한 구현과 역할을 살펴보겠습니다. 피드포워드 네트워크는 트랜스포머의 표현력을 크게 향상시키는 핵심 구성 요소입니다.

---

## 네비게이션


- **이전**: 인코더 아키텍처
- **다음**: 피드포워드 네트워크
- **목차**: README.md
- **전체 목록**: 네비게이션


**관련 문서:**

- 기초: 어텐션 메커니즘
- 구성 요소: 멀티헤드 어텐션
- 구성 요소: 포지셔널 인코딩